

Below is a **150-question Full-Stack Developer interview set** focused on **Node.js, NestJS, React.js, JavaScript/TypeScript, databases, APIs, system design, security, testing, DevOps, and live coding**.

I kept the answers in **simple English**, with a natural length for a remote interview. You can use them as speaking answers rather than memorizing long textbook definitions.

Part 1 — JavaScript & TypeScript

1. What is the difference between `var`, `let`, and `const`?

`var` has function scope, while `let` and `const` have block scope. `let` allows us to change the value later. `const` does not allow us to assign a new value. In modern JavaScript, I normally use `const` by default and `let` when the value needs to change.

2. What is the difference between `==` and `===`?

`==` compares values and can convert their types before comparing them. `===` compares both the value and the type. I normally use `===` because it is safer and more predictable.

3. What is a closure?

A closure happens when a function remembers variables from its outer scope, even after the outer function has finished. Closures are useful for private data, callbacks, and creating functions with their own state.

4. What is hoisting?

Hoisting means JavaScript moves some declarations to the top of their scope during execution setup. Function declarations can be used before they are written. Variables declared with `var` are also hoisted, but `let` and `const` cannot be safely used before their declaration.

5. What is the difference between synchronous and asynchronous code?

Synchronous code runs one step at a time and waits for each operation to finish. Asynchronous code can start an operation and continue doing other work while waiting. In Node.js, asynchronous programming is very important for database queries, HTTP requests, and file operations.

6. What is a Promise?

A Promise represents the future result of an asynchronous operation. It can be in a pending, fulfilled, or rejected state. I usually use `async/await` because it makes asynchronous code easier to read.

7. What is `async/await`?

`async/await` is a cleaner way to work with Promises. An `async` function returns a Promise, and `await` waits for a Promise to finish. It makes asynchronous code look more like normal synchronous code.

8. What is the difference between `Promise.all()` and `Promise.allSettled()`?

`Promise.all()` fails when one of the Promises fails. It is useful when all operations must succeed.

`Promise.allSettled()` waits for all operations and gives us the result of each one, even if some of them fail.

9. What is the JavaScript event loop?

The event loop allows JavaScript to handle asynchronous operations without blocking the main thread. JavaScript runs the current code first, and when asynchronous operations are ready, their callbacks are added to queues and processed by the event loop.

10. Is Node.js single-threaded?

The JavaScript execution part of Node.js runs mainly on a single thread. But Node.js uses the operating system and a thread pool for some background operations. Node.js can also use worker threads and multiple processes when we need more CPU power.

11. What is the difference between `map`, `filter`, and `reduce`?

`map` creates a new array by changing every item. `filter` creates a new array with only the items that match a condition. `reduce` processes all items and produces one final value, such as a sum or an object.

12. What is destructuring?

Destructuring allows us to extract values from objects or arrays easily. For example, instead of accessing `user.name` and `user.email` separately, we can extract both values in one statement.

13. What is the spread operator?

The spread operator `...` allows us to copy or combine arrays and objects. For example, we can create a new object with `{ ...user, name: 'John' }`. It is also useful for avoiding direct changes to existing data.

14. What is the difference between shallow copy and deep copy?

A shallow copy copies the first level of an object, but nested objects are still shared. A deep copy also copies nested objects. This is important when we work with complex objects and want to avoid unexpected changes.

15. What is immutability?

Immutability means we do not directly change existing data. Instead, we create a new version with the required changes. This is very important in React because it helps React detect state changes correctly.

16. What is TypeScript?

TypeScript is a superset of JavaScript that adds static typing and other features. It helps us find many errors before running the application. I use TypeScript because it makes large applications easier to maintain.

17. What is the difference between `interface` and `type`?

Both can describe the structure of data. Interfaces are often used for object structures and can be extended. Types are more flexible because they can describe unions, intersections, primitives, and other structures.

18. What are generics in TypeScript?

Generics allow us to write reusable code that works with different types while keeping type safety. For example, a generic response class can work with users, orders, or products without losing type information.

19. What is an enum?

An enum defines a fixed set of values. For example, an order status can be `PENDING`, `PAID`, or `CANCELLED`. I use enums when a value should only come from a known list.

20. What is type narrowing?

Type narrowing means TypeScript can understand a more specific type after we check it. For example, after checking `typeof value === 'string'`, TypeScript knows that the value is a string.

Part 2 — Node.js

21. Why is Node.js popular for backend development?

Node.js uses JavaScript and has strong support for asynchronous operations. It works very well for APIs, real-time applications, and I/O-heavy systems. It also has a very large package ecosystem through npm.

22. What is npm?

npm is the package manager for Node.js. We use it to install libraries, manage dependencies, run scripts, and publish packages.

23. What is `package.json`?

`package.json` contains project information, scripts, dependencies, and configuration. It is one of the main files in a Node.js project.

24. What is the difference between dependencies and devDependencies?

`dependencies` are packages needed when the application runs. `devDependencies` are mainly needed during development, testing, or building, such as testing tools and TypeScript tools.

25. What is middleware?

Middleware is code that runs between receiving a request and sending a response. It can be used for logging, authentication, validation, headers, and other common operations.

26. What is a REST API?

REST is an architectural style for building web APIs. It usually uses HTTP methods such as GET, POST, PUT, PATCH, and DELETE. Resources are normally represented by URLs.

27. What is the difference between PUT and PATCH?

PUT normally replaces the whole resource. PATCH updates part of the resource. For example, if I only want to change a user's email, PATCH is usually more appropriate.

28. What are HTTP status codes?

HTTP status codes tell the client what happened with the request. For example, 200 means success, 201 means created, 400 means bad request, 401 means unauthenticated, 403 means forbidden, and 404 means not found.

29. What is CORS?

CORS controls which origins are allowed to access a server from a browser. It is a browser security mechanism. We configure it on the backend to allow trusted frontend applications.

30. What is a stream in Node.js?

A stream allows us to process data piece by piece instead of loading everything into memory. Streams are useful for large files, uploads, downloads, and other large data operations.

31. What is a Buffer?

A Buffer is used to work with binary data in Node.js. It is common when working with files, network data, images, and streams.

32. What is the Node.js cluster?

The cluster module allows us to run multiple Node.js processes so we can use multiple CPU cores. A load balancer can distribute requests between these processes.

33. What are worker threads?

Worker threads allow us to run CPU-heavy JavaScript operations in separate threads. They are useful when a task would otherwise block the main Node.js thread.

34. What can cause memory leaks in Node.js?

Common causes include global variables, objects that are kept in memory unnecessarily, event listeners that are never removed, and large caches. Memory leaks can slowly increase memory usage and eventually crash the application.

35. How do you handle errors in Node.js?

I normally use structured error handling with `try/catch` for asynchronous code and centralized error handling for HTTP requests. I also log important errors with enough context to debug them.

Part 3 — NestJS

36. What is NestJS?

NestJS is a Node.js framework built mainly with TypeScript. It provides modules, controllers, services, dependency injection, guards, interceptors, pipes, and other features that help us build large backend applications.

37. What is a module in NestJS?

A module groups related functionality together. It can contain controllers, services, providers, and imported modules. Good module design helps keep the application organized.

38. What is a controller?

A controller handles incoming HTTP requests. It receives parameters and data from the client and calls services to perform the business logic.

39. What is a service?

A service contains business logic or reusable application logic. Controllers should normally stay simple and delegate the main work to services.

40. What is dependency injection?

Dependency injection means a class receives the objects it needs instead of creating them itself. NestJS provides a dependency injection container that manages these dependencies.

41. What is a DTO?

DTO means Data Transfer Object. It defines the structure of data that enters or leaves an application layer. In NestJS, DTOs are often used with validation.

42. What are pipes in NestJS?

Pipes can transform and validate incoming data. For example, `ValidationPipe` can check whether a request body matches the DTO rules.

43. What are guards?

Guards decide whether a request is allowed to continue. They are commonly used for authentication and authorization.

44. What are interceptors?

Interceptors can run before and after a request handler. They are useful for logging, transforming responses, measuring execution time, caching, and other cross-cutting concerns.

45. What are exception filters?

Exception filters provide centralized handling for exceptions. They allow us to control the HTTP response when an error occurs.

46. What is a custom decorator in NestJS?

A custom decorator allows us to create reusable metadata or request-related logic. For example, we can create a `@CurrentUser()` decorator to get the logged-in user.

47. What is the difference between middleware and guards?

Middleware runs before the route handler and is not mainly designed for authorization decisions. Guards are integrated with NestJS routing and are designed to decide whether a request can continue.

48. How would you structure a large NestJS project?

I would organize the project by business modules. For example, `users`, `orders`, `products`, and `payments`. Inside each module, I would separate controllers, services, DTOs, entities, repositories, and other related code.

49. How do you validate requests in NestJS?

I usually use DTOs with `class-validator` and NestJS's `ValidationPipe`. I can also enable options such as `whitelist` to remove unexpected properties.

50. How do you handle configuration in NestJS?

I normally use `ConfigModule` and environment variables. Sensitive values such as database passwords and secret keys should not be hardcoded in the source code.

Part 4 — Databases & SQL

51. What is the difference between SQL and NoSQL?

SQL databases use structured tables and relationships. NoSQL databases use models such as documents, key-value data, or wide columns. I choose based on the application's data and access patterns.

52. What is PostgreSQL?

PostgreSQL is a relational database with strong SQL support, transactions, indexes, constraints, JSON support, and many advanced features. It is a strong choice for many business applications.

53. What is a primary key?

A primary key uniquely identifies a record in a table. It should be unique and normally should not be null.

54. What is a foreign key?

A foreign key creates a relationship between two tables. It usually points to the primary key of another table.

55. What is normalization?

Normalization organizes database tables to reduce duplicate data and improve data consistency. It usually means separating related data into different tables with relationships.

56. What is denormalization?

Denormalization intentionally duplicates some data to improve read performance or simplify queries. It can make writes and data consistency more difficult.

57. What is an index?

An index helps the database find rows faster. However, indexes also use storage and can make insert and update operations slower, so we should add them based on real query patterns.

58. What is a database transaction?

A transaction groups multiple database operations into one logical operation. If something fails, we can roll back the changes. This helps keep data consistent.

59. What are ACID properties?

ACID means Atomicity, Consistency, Isolation, and Durability. These properties help databases safely handle transactions.

60. What is a database deadlock?

A deadlock happens when two or more transactions wait for each other and cannot continue. We can reduce deadlocks with good transaction design, consistent lock order, and short transactions.

61. What is a JOIN?

A JOIN combines data from multiple tables based on a relationship. Common types are **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, and **FULL JOIN**.

62. What is an N+1 query problem?

It happens when we make one query to get a list and then make another query for each item. This can create many database queries and cause performance problems. We can solve it with joins, eager loading, batching, or better query design.

63. What is database pagination?

Pagination means returning data in smaller pages instead of returning everything. Common approaches are offset pagination and cursor pagination.

64. Offset vs cursor pagination?

Offset pagination uses values such as page number and limit. It is simple but can become slower for large datasets. Cursor pagination uses a position or unique key and is usually better for large or frequently changing datasets.

65. How do you improve a slow database query?

First, I check the query and execution plan. Then I look for missing indexes, unnecessary joins, large data scans, and inefficient conditions. I would optimize based on actual database measurements.

Part 5 — API Design & Security

66. What is authentication?

Authentication answers the question: "Who are you?" For example, a user logs in with an email and password.

67. What is authorization?

Authorization answers the question: "What are you allowed to do?" For example, an admin may delete users while a normal user cannot.

68. What is JWT?

JWT is a token format commonly used for authentication. The server signs the token, and the client sends it with later requests. The server can verify the token without storing the full session state.

69. Where should we store JWT tokens?

There are different approaches. For browser applications, secure, HTTP-only cookies are often a good choice because JavaScript cannot directly access them. The final design depends on the application's security and architecture.

70. What is refresh token rotation?

Instead of using the same refresh token for a long time, the server gives a new refresh token when the old one is used. The old token becomes invalid. This can reduce the impact of a stolen refresh token.

71. How should passwords be stored?

Passwords should never be stored as plain text. We should use a strong password hashing algorithm such as Argon2 or bcrypt with an appropriate configuration.

72. What is SQL injection?

SQL injection happens when user input is inserted into SQL queries in an unsafe way. Parameterized queries or ORM query builders can help prevent it.

73. What is XSS?

XSS means Cross-Site Scripting. It happens when an attacker can inject malicious JavaScript into a web page. Proper output escaping, safe rendering, and Content Security Policy can help reduce the risk.

74. What is CSRF?

CSRF tricks a user's browser into sending an unwanted request to a website where the user is authenticated. SameSite cookies and CSRF tokens are common protections.

75. What is rate limiting?

Rate limiting controls how many requests a client can make in a period of time. It helps protect APIs from abuse, brute-force attacks, and excessive traffic.

76. What is throttling?

Throttling limits how frequently an operation can happen. For example, we may allow only a certain number of login attempts within a specific period.

77. How do you secure an API?

I would use authentication, authorization, input validation, HTTPS, rate limiting, secure headers, safe password hashing, proper error handling, logging, and least-privilege access.

78. What is API versioning?

API versioning allows us to change an API without immediately breaking existing clients. For example, we can have `/api/v1/users` and `/api/v2/users`.

79. What is idempotency?

An operation is idempotent when sending the same request multiple times produces the same final result. This is very important for payment and order APIs where a request might be retried.

80. What is an idempotency key?

An idempotency key is a unique key sent with a request. The server stores the result for that key and returns the same result if the client sends the request again.

Part 6 — React.js

81. What is React?

React is a JavaScript library for building user interfaces. It uses components and a declarative approach to describe how the UI should look.

82. What is a React component?

A component is a reusable piece of UI and logic. It can receive data through props and manage its own state.

83. What are props?

Props are values passed from a parent component to a child component. They are read-only from the child's point of view.

84. What is state?

State is data managed by a component that can change over time. When state changes, React can render the component again.

85. What is `useState`?

`useState` is a React Hook used to store local component state. It returns the current value and a function that can update it.

86. What is `useEffect`?

`useEffect` is used for side effects such as API calls, subscriptions, timers, and interacting with external systems. It runs after rendering based on its dependency array.

87. What is the dependency array in `useEffect`?

The dependency array tells React when the effect should run again. If a value used by the effect changes, React can run the effect again.

88. What is `useMemo`?

`useMemo` stores the result of a calculation and can prevent repeating an expensive calculation when the dependencies have not changed. It should be used when there is a real performance reason.

89. What is `useCallback`?

`useCallback` stores a function reference between renders. It can be useful when passing functions to memoized child components.

90. What is `React.memo`?

`React.memo` can prevent a component from rendering again when its props have not changed. It is mainly useful for performance optimization.

91. What is the Virtual DOM?

The Virtual DOM is a representation of the UI that React uses to determine what needs to change. React compares the new and previous representations and updates the real DOM where needed.

92. What is reconciliation?

Reconciliation is the process React uses to compare the previous UI tree with the new one and decide what changes are needed.

93. Why are keys important in React lists?

Keys help React identify which items changed, were added, or were removed. A stable unique ID is normally better than using the array index as a key.

94. What is controlled input?

A controlled input gets its value from React state. When the user types, an event updates the state. This gives React full control over the input value.

95. What is an uncontrolled input?

An uncontrolled input stores its value in the DOM instead of React state. We can access the value using a ref when needed.

96. What is lifting state up?

Lifting state up means moving shared state to the nearest common parent component. This allows multiple child components to use the same source of truth.

97. What is Context API?

Context allows us to share data with many components without passing props through every level. It can be useful for themes, authentication information, and other global or shared data.

98. When would you use Redux?

I would use Redux when application state is large, shared across many parts of the application, or has complex update logic. For simple local state, React state and Context may be enough.

99. What is React Query or TanStack Query?

TanStack Query is used mainly for server state. It helps with fetching, caching, synchronization, retries, loading states, and refetching API data.

100. How do you improve React performance?

I first find the actual bottleneck using profiling tools. Then I may reduce unnecessary renders, use memoization when useful, split large components, lazy-load code, optimize images, and improve API calls.

Part 7 — Testing

101. What is unit testing?

Unit testing tests a small piece of code in isolation, such as a function or service. The goal is to verify that the unit behaves correctly.

102. What is integration testing?

Integration testing checks whether multiple parts work together correctly. For example, we can test a service together with a real or test database.

103. What is end-to-end testing?

E2E testing tests the complete application flow from the user's point of view. For example, we can test login, product selection, and order creation.

104. What is mocking?

Mocking means replacing a real dependency with a fake implementation. For example, while testing a service, we can mock the database repository.

105. What should we test in a NestJS service?

I would test the business rules, successful operations, validation errors, not-found cases, dependency failures, and important edge cases.

106. What is test coverage?

Test coverage shows how much of the code is executed by tests. High coverage is useful, but high coverage alone does not mean the tests are good.

107. What is TDD?

TDD means Test-Driven Development. The common cycle is **Red, Green, Refactor**: write a failing test, make it pass, and then improve the code.

108. What is the difference between unit and E2E tests?

Unit tests are fast and test small pieces of code. E2E tests are slower but test real application flows. A good project normally uses both.

109. How do you test an API?

I test successful requests, invalid input, authentication, authorization, error responses, edge cases, and important business rules. Tools such as Jest, Supertest, or Playwright can help.

110. How do you make tests reliable?

Tests should be independent and should not depend on execution order. I also avoid unnecessary external dependencies and clean up test data after each test.

Part 8 — Architecture & Design Patterns

111. What is SOLID?

SOLID is a group of five design principles that help make software easier to maintain. They are Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.

112. What is the Single Responsibility Principle?

A class or module should have one main responsibility. For example, a service that handles orders should not also handle email templates and file storage.

113. What is dependency inversion?

High-level business logic should not directly depend on low-level implementations. Both should depend on abstractions. This makes the code easier to test and change.

114. What is clean architecture?

Clean Architecture separates business rules from frameworks, databases, and external systems. The main idea is that business logic should not strongly depend on infrastructure.

115. What is DDD?

DDD means Domain-Driven Design. It focuses on modeling software around the business domain and its rules. It is useful when the business logic is complex.

116. What is CQRS?

CQRS means Command Query Responsibility Segregation. Commands change data, while queries read data. This separation can make complex systems easier to design and scale.

117. What is repository pattern?

A repository hides database access behind an abstraction. Business logic can work with the repository without knowing the exact database implementation.

118. What is the service layer?

The service layer contains application or business logic between controllers and infrastructure. It prevents controllers from becoming too large.

119. What is a monolith?

A monolith is an application where most or all functionality is deployed as one unit. It can be simple to develop and deploy, especially for smaller systems.

120. What is a microservice?

A microservice architecture divides an application into smaller services. Each service normally owns a specific business capability and can be deployed independently.

Part 9 — System Design

121. How would you design a URL shortener?

I would create an API that receives a long URL and generates a unique short code. I would store the code and original URL in a database. Redis could cache popular URLs, and the service could use multiple application instances behind a load balancer.

122. How would you design a scalable REST API?

I would start with stateless application servers behind a load balancer. I would use a database with proper indexes, caching where useful, pagination, rate limiting, logging, monitoring, and horizontal scaling.

123. How would you design an e-commerce system?

I would separate important domains such as users, products, orders, payments, and inventory. For a smaller system, they can start as modules in one application. If scale or team structure requires it, some modules can later become services.

124. How would you design an order system?

I would have entities such as Order, OrderItem, Product, Customer, and Payment. The order would have a clear state flow such as **PENDING**, **CONFIRMED**, **SHIPPED**, and **CANCELLED**. Important operations would use transactions and idempotency.

125. How would you prevent duplicate orders?

I would use an idempotency key for the create-order request. I would also use database constraints and transactions where needed. This protects the system if the client retries a request.

126. How would you design a payment system?

I would keep payment logic separate from the order logic. I would store payment states and external transaction IDs. Webhooks from the payment provider should be verified and processed safely and idempotently.

127. How would you design a notification system?

I would create a notification service that accepts events such as **ORDER_CREATED**. A message broker such as Kafka or RabbitMQ could distribute these events. Separate workers could send email, SMS, or push notifications.

128. Why use a message broker?

A message broker allows services to communicate asynchronously. It can reduce direct dependency between services, handle traffic spikes, and support event-driven architecture.

129. Kafka vs RabbitMQ?

Kafka is very strong for high-throughput event streaming and keeping event history. RabbitMQ is often simpler for traditional message queues and routing. The right choice depends on the system's requirements.

130. What is caching?

Caching stores frequently used data in a faster location. Redis is a common caching solution. The main challenge is keeping cached data fresh when the original data changes.

131. What is cache invalidation?

Cache invalidation means removing or updating cached data when the original data changes. It is important because stale cache data can cause incorrect results.

132. What is horizontal scaling?

Horizontal scaling means adding more application instances. A load balancer distributes traffic between them. It is usually a common way to scale stateless APIs.

133. What is vertical scaling?

Vertical scaling means giving one server more CPU, RAM, or other resources. It is simple, but there is a physical limit and it can become expensive.

134. How would you handle high traffic?

I would first measure the bottleneck. Then I might use horizontal scaling, caching, database indexes, read replicas, queues, CDN, pagination, and rate limiting. I would not add complex infrastructure without understanding the actual bottleneck.

135. How would you design file uploads?

For large files, I would avoid sending the entire file through the application server if possible. The backend can generate a temporary upload URL, and the client can upload directly to object storage such as S3. The backend then stores the file metadata.

136. How would you design a chat application?

I would use WebSockets for real-time communication. Messages could be stored in a database, while Redis could help with presence and scaling. For larger systems, a message broker can distribute events between WebSocket servers.

137. How would you design a search system?

For simple search, database indexes may be enough. For advanced full-text search, I could use Elasticsearch or OpenSearch. I would index searchable fields and update the index when the source data changes.

138. How would you handle distributed transactions?

I would avoid distributed transactions when possible. For microservices, patterns such as Saga and transactional outbox can help coordinate operations while keeping services independent.

139. What is the transactional outbox pattern?

The application writes the business data and an event record into the same database transaction. A separate process reads the outbox and publishes the event. This helps prevent a situation where database data is saved but the event is lost.

140. What are observability and monitoring?

Observability helps us understand what is happening inside a system. I would use logs, metrics, and distributed traces. Tools such as Prometheus, Grafana, and OpenTelemetry can help.

Part 10 — Live Coding Questions

141. Write a function to reverse a string.

A simple solution is to split the string into characters, reverse the array, and join it again.

```
function reverseString(value: string): string {  
  return value.split("").reverse().join("");  
}
```

```
}
```

This is simple and has $O(n)$ time complexity.

142. Write a function to check if a string is a palindrome.

I would compare the string with its reversed version.

```
function isPalindrome(value: string): boolean {  
  const reversed = value.split("").reverse().join("");  
  return value === reversed;  
}
```

For a simple interview question, this solution is clear and easy to understand.

143. Find duplicate numbers in an array.

I can use a **Set** to keep track of numbers that I have already seen.

```
function findDuplicates(numbers: number[]): number[] {  
  const seen = new Set<number>();  
  const duplicates = new Set<number>();  
  
  for (const number of numbers) {  
    if (seen.has(number)) {  
      duplicates.add(number);  
    }  
  
    seen.add(number);  
  }  
  
  return [...duplicates];  
}
```

The time complexity is approximately $O(n)$.

144. Find the first non-repeated character.

I would first count each character and then check the string again.

```
function firstUniqueChar(value: string): string | null {  
  const counts = new Map<string, number>();  
  
  for (const char of value) {  
    counts.set(char, (counts.get(char) ?? 0) + 1);  
  }  
}
```



```
for (const char of value) {
  if (counts.get(char) === 1) {
    return char;
  }
}

return null;
}
```

This solution is easy to understand and works in $O(n)$ time.

145. Implement a debounce function.

Debounce means we wait until the user stops calling a function for a specific amount of time.

```
function debounce(
  callback: (...args: any[]) => void,
  delay: number,
) {
  let timer: ReturnType<typeof setTimeout>;

  return (...args: any[]) => {
    clearTimeout(timer);

    timer = setTimeout(() => {
      callback(...args);
    }, delay);
  };
}
```

It is commonly used for search inputs so we do not send an API request for every keystroke.

146. Implement a simple throttle function.

Throttle limits how often a function can run.

```
function throttle(
  callback: (...args: any[]) => void,
  delay: number,
) {
  let lastCall = 0;

  return (...args: any[]) => {
    const now = Date.now();

    if (now - lastCall >= delay) {
      lastCall = now;
      callback(...args);
    }
  }
}
```

```
};  
}
```

This can be useful for scroll, resize, or mouse events.

147. Create a simple API endpoint in NestJS.

I would create a controller and service. For example:

```
@Controller('users')  
export class UsersController {  
  constructor(private readonly userService: UsersService) {}  
  
  @Get(':id')  
  async getUser(@Param('id') id: string) {  
    return this.userService.findById(id);  
  }  
}
```

The controller receives the request and delegates the business logic to the service.

148. How would you implement a React search input with an API call?

I would keep the search value in state and use a debounce mechanism before calling the API.

```
const [search, setSearch] = useState("");  
  
const handleSearch = (value: string) => {  
  setSearch(value);  
};  
  
return (  
  <input  
    value={search}  
    onChange={(e) => handleSearch(e.target.value)}  
  />  
);
```

In a real application, I would usually use a debounced function or a library such as TanStack Query to manage the API request and caching.

149. How would you solve a slow API endpoint during a live coding interview?

First, I would try to understand where the time is spent. I would check the database query, external API calls, CPU-heavy operations, and network time. Then I would use profiling or logs to find the real bottleneck instead of guessing.

For example, if the database query is slow, I may add an index or improve the query. If an external call is slow, I may use caching or parallel requests where it is safe.

150. How would you design and implement a simple CRUD API?

I would first define the resource and its data model. Then I would create DTOs, a controller, a service, and a repository or ORM layer.

For example:

```
POST /users
GET /users
GET /users/:id
PATCH /users/:id
DELETE /users/:id
```

I would also add validation, authentication where required, proper HTTP status codes, error handling, database constraints, and tests. For a production system, I would also think about pagination, logging, authorization, and rate limiting.

Quick Interview Strategy

For a **Full-Stack Node.js/NestJS/React.js** interview, I would prepare these areas especially well:

Area	What you should be able to explain
JavaScript	Async/await, Promise, event loop, closures, array methods
TypeScript	Types, interfaces, generics, enums, narrowing
Node.js	Event loop, streams, workers, error handling
NestJS	Modules, DI, guards, pipes, interceptors, DTOs
REST API	HTTP methods, status codes, pagination, versioning
Security	JWT, OAuth concepts, XSS, CSRF, SQL injection
PostgreSQL	Joins, indexes, transactions, normalization
React	Hooks, state, props, effects, rendering
State Management	Context, Redux, TanStack Query
Testing	Unit, integration, E2E, mocking, TDD
Architecture	SOLID, DDD, CQRS, clean architecture

Microservices	Kafka, queues, events, Saga, outbox
System Design	Scaling, caching, load balancing, databases
Live Coding	Arrays, strings, async code, API implementation
DevOps	Docker, CI/CD, logging, monitoring

A good interview answer does not need to sound like a textbook. A strong pattern is:

Definition → How I use it → Small example → Trade-off if needed

For example, instead of giving a long explanation of caching:

"Caching means keeping frequently used data in a faster place, like Redis. I use it when the same data is requested many times and does not change very often. It can reduce database load and improve response time. The main problem is cache invalidation, because we need to make sure users do not get old data."

That style usually sounds much more natural in a remote technical interview.